

**LAPORAN PRAKTIKUM PEMBELAJARAN MESIN: IMPLEMENTASI
CONVOLUTIONAL NEURAL NETWORK (CNN) DAN *TRANSFER LEARNING*
UNTUK KLASIFIKASI CITRA
PENGOLAHAN CITRA DIGITAL**



Dosen Pengampu:
Dr. Yasdinul Huda, S.Pd., M.T.

Oleh:
Muhammad Zahran
24343077

**PROGRAM STUDI INFORMATIKA
DEPARTEMEN TEKNIK ELEKTRONIKA
FAKULTAS TEKNIK
UNIVERSITAS NEGERI PADANG
2026**

A. Pendahuluan

Perkembangan *Deep Learning*, khususnya arsitektur *Convolutional Neural Network* (CNN), telah membawa kemajuan pesat dalam bidang visi komputer (*computer vision*). Arsitektur ini sangat efektif dalam mengenali pola spasial pada data berbasis piksel. Praktikum ini bertujuan untuk mengeksplorasi dua pendekatan utama dalam klasifikasi citra: membangun model CNN dari awal (*from scratch*) untuk mengklasifikasikan dataset publik, serta memanfaatkan model pra-latih (*pre-trained model*) melalui teknik *Transfer Learning* dan augmentasi data guna meningkatkan efisiensi dan performa model pada dataset terbatas.

B. Metodologi dan Implementasi

Praktikum ini dibagi menjadi dua eksperimen utama yang dijalankan menggunakan pustaka TensorFlow dan Keras.

1. Latihan 1: Membangun CNN dari Awal untuk Dataset CIFAR-10

Pada tahap pertama, model dibangun untuk mengklasifikasikan dataset CIFAR-10 yang terdiri dari 10 kelas objek (seperti pesawat, mobil, burung, dll).

- **Prapemrosesan:** Citra dinormalisasi agar nilai piksel berada pada rentang 0 hingga 1. Label data diubah ke dalam format *one-hot encoding* agar sesuai dengan fungsi *loss categorical_crossentropy*.
- **Arsitektur Model:** Model dibangun menggunakan Keras Functional API dengan tiga blok konvolusi utama. Setiap blok terdiri dari lapisan *Conv2D* untuk ekstraksi fitur, *Batch Normalization* untuk menstabilkan proses pelatihan, *MaxPooling2D* untuk reduksi dimensi, dan *Dropout* untuk mencegah *overfitting*. Fitur yang diekstraksi kemudian diratakan (*Flatten*) dan diteruskan ke lapisan padat (*Dense*) untuk menghasilkan prediksi probabilitas multikelas.
- **Strategi Pelatihan:** Model dilatih menggunakan optimasi Adam. Untuk menjaga efisiensi, digunakan *callbacks* berupa *EarlyStopping* (menghentikan pelatihan jika akurasi validasi tidak meningkat) dan *ReduceLROnPlateau* (menurunkan *learning rate* secara dinamis jika model kesulitan menemukan titik optimal).

2. Latihan 2: *Transfer Learning* dan Augmentasi Data

Tahap kedua mendemonstrasikan bagaimana model yang telah dilatih pada dataset raksasa (ImageNet) dapat digunakan kembali untuk tugas klasifikasi biner baru

dengan data terbatas. Dataset yang digunakan adalah data sintetik (lingkaran dan segitiga) yang merepresentasikan kelas kucing dan anjing.

- **Augmentasi Data:** Untuk memperkaya variasi data latih, diterapkan lapisan augmentasi yang memberikan efek acak seperti pembalikan horizontal (*flip*), rotasi, perbesaran (*zoom*), dan pergeseran posisi.
- **Ekstraksi Fitur (Transfer Learning):** Model dasar menggunakan VGG16. Lapisan konvolusi dari VGG16 dibekukan (*frozen*) sehingga bobotnya tidak berubah selama pelatihan awal. Lapisan klasifikasi di bagian ujung diganti dengan lapisan *Dense* baru yang diakhiri dengan aktivasi *sigmoid* untuk klasifikasi biner. Eksperimen ini membandingkan langsung performa model yang menggunakan data augmentasi dengan yang tidak.
- **Penyesuaian Tahap Akhir (Fine-Tuning):** Setelah pelatihan awal, 15 lapisan terakhir dari VGG16 "dicairkan" (*unfrozen*) dan dilatih kembali menggunakan *learning rate* yang sangat kecil. Tujuannya adalah membiarkan model beradaptasi secara spesifik terhadap detail fitur pada dataset baru tanpa merusak bobot fundamental yang sudah bagus.
- **Komparasi Model:** Sebagai langkah akhir, dilakukan perbandingan performa dan ukuran parameter antara berbagai arsitektur populer, yaitu VGG16, ResNet50, MobileNetV2, dan EfficientNetB0.

C. Hasil dan Pembahasan

1. Analisis Latihan 1 (CNN CIFAR-10)

Penggunaan kombinasi *Batch Normalization* dan *Dropout* terbukti efektif dalam menjaga keseimbangan antara performa pada data latih dan data validasi (mengurangi *overfitting*). Selain itu, visualisasi peta fitur (*feature maps*) pada lapisan konvolusi awal menunjukkan bagaimana model secara hierarkis belajar mengenali fitur visual sederhana, seperti tepi dan gradien warna, sebelum memproses pola yang lebih kompleks di lapisan berikutnya.

2. Analisis Latihan 2 (Transfer Learning)

Dari hasil komparasi, model yang menggunakan **augmentasi data** menunjukkan kurva akurasi dan *loss* validasi yang lebih stabil dibandingkan model tanpa augmentasi, membuktikan bahwa augmentasi membantu model menggeneralisasi data dengan lebih baik.

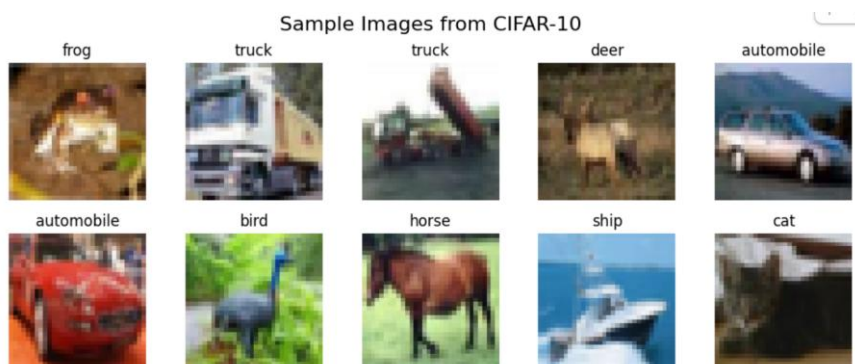
Evaluasi menggunakan matriks performa komprehensif (seperti *Kurva Precision-Recall*, matriks konfusi, dan kurva ROC) menunjukkan bahwa model transfer learning sangat tangguh. Proses *fine-tuning* terbukti mampu memberikan peningkatan akurasi tambahan (*improvement*), karena fitur tingkat tinggi (*high-level features*) pada model pra-latih berhasil disesuaikan dengan karakteristik unik dari data sintetik.

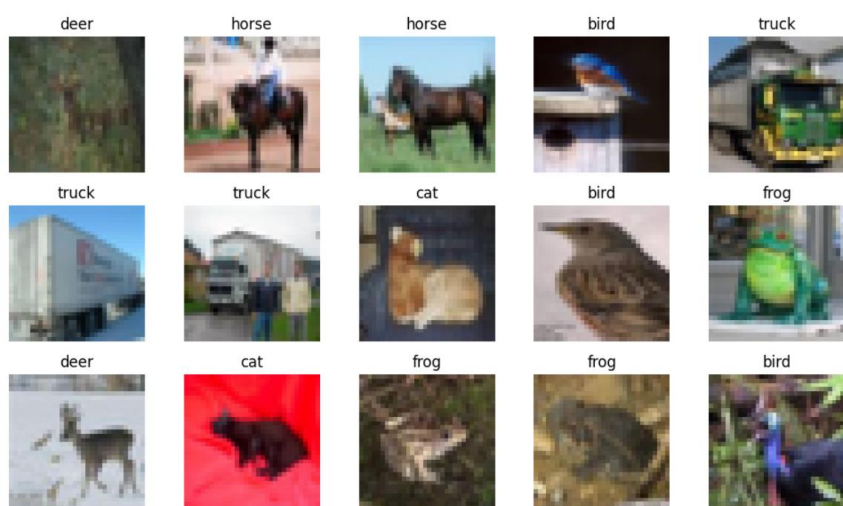
Pada uji komparasi antar-arsitektur (VGG16 vs ResNet50 vs MobileNetV2 vs EfficientNetB0), terlihat adanya *trade-off* (kompromi) antara ukuran model dan akurasi. Model seperti MobileNetV2 atau EfficientNetB0 cenderung menawarkan efisiensi komputasi yang jauh lebih baik (jumlah parameter lebih sedikit) namun tetap mempertahankan akurasi yang sangat kompetitif, menjadikannya ideal untuk diimplementasikan pada perangkat dengan sumber daya terbatas.

D. Kesimpulan

1. Membangun CNN dari awal memerlukan desain arsitektur yang hati-hati, di mana *Batch Normalization* dan *Dropout* merupakan komponen krusial untuk mencegah *overfitting*.
2. *Transfer Learning* mempercepat proses pelatihan dan memberikan hasil yang sangat baik meskipun data latih terbatas, karena model mendaur ulang pengetahuan dari dataset raksasa.
3. Penerapan augmentasi data mencegah model menghafal data latih, sementara teknik *fine-tuning* berfungsi untuk menyempurnakan adaptasi model terhadap masalah klasifikasi yang spesifik.

E. Lampiran Output





CNN MODEL ARCHITECTURE:

=====

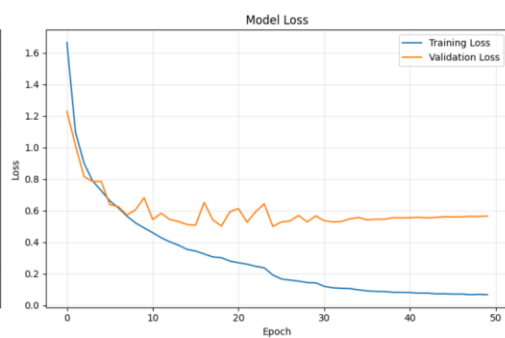
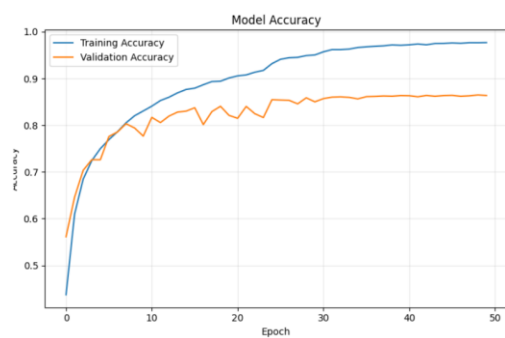
Model: "functional_2"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 32, 32, 3)	0
conv1 (Conv2D)	(None, 32, 32, 32)	896
bn1 (BatchNormalization)	(None, 32, 32, 32)	128
conv2 (Conv2D)	(None, 32, 32, 32)	9,248
bn2 (BatchNormalization)	(None, 32, 32, 32)	128
pool1 (MaxPooling2D)	(None, 16, 16, 32)	0
dropout1 (Dropout)	(None, 16, 16, 32)	0
conv3 (Conv2D)	(None, 16, 16, 64)	18,496
bn3 (BatchNormalization)	(None, 16, 16, 64)	256
conv4 (Conv2D)	(None, 16, 16, 64)	36,928
bn4 (BatchNormalization)	(None, 16, 16, 64)	256
pool2 (MaxPooling2D)	(None, 8, 8, 64)	0
dropout2 (Dropout)	(None, 8, 8, 64)	0
conv5 (Conv2D)	(None, 8, 8, 128)	73,856
bn5 (BatchNormalization)	(None, 8, 8, 128)	512
conv6 (Conv2D)	(None, 8, 8, 128)	147,584
bn6 (BatchNormalization)	(None, 8, 8, 128)	512
pool3 (MaxPooling2D)	(None, 4, 4, 128)	0
dropout3 (Dropout)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
fc1 (Dense)	(None, 256)	524,544
bn7 (BatchNormalization)	(None, 256)	1,024
dropout4 (Dropout)	(None, 256)	0
output (Dense)	(None, 10)	2,570

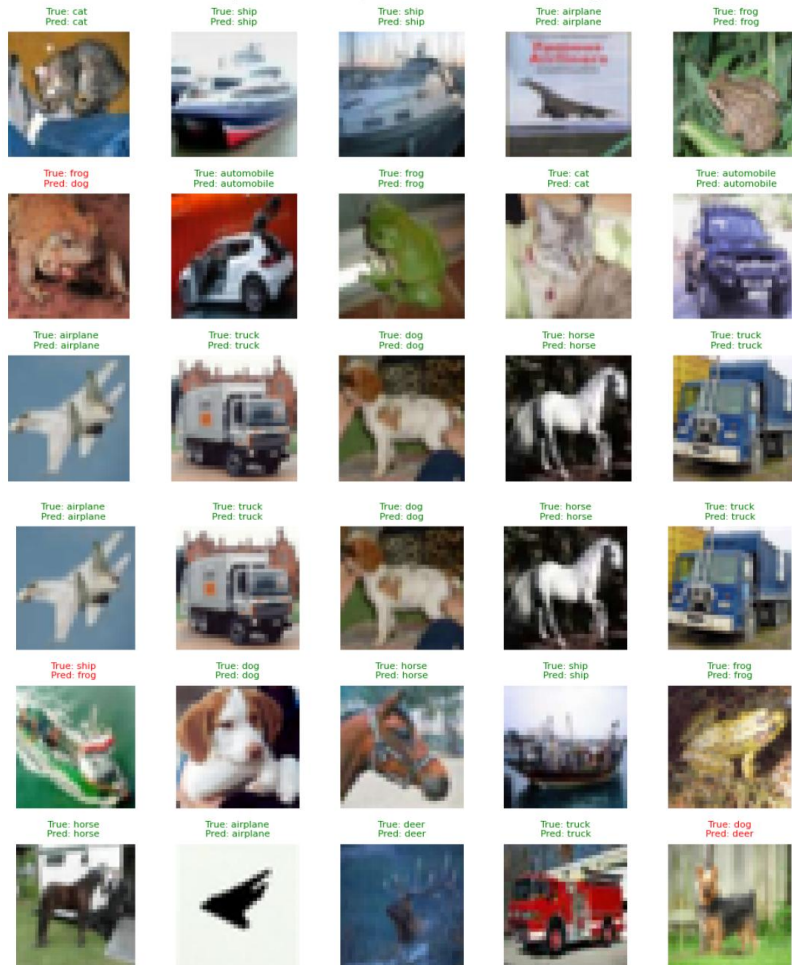
Total params: 816,938 (3.12 MB)

Trainable params: 815,530 (3.11 MB)

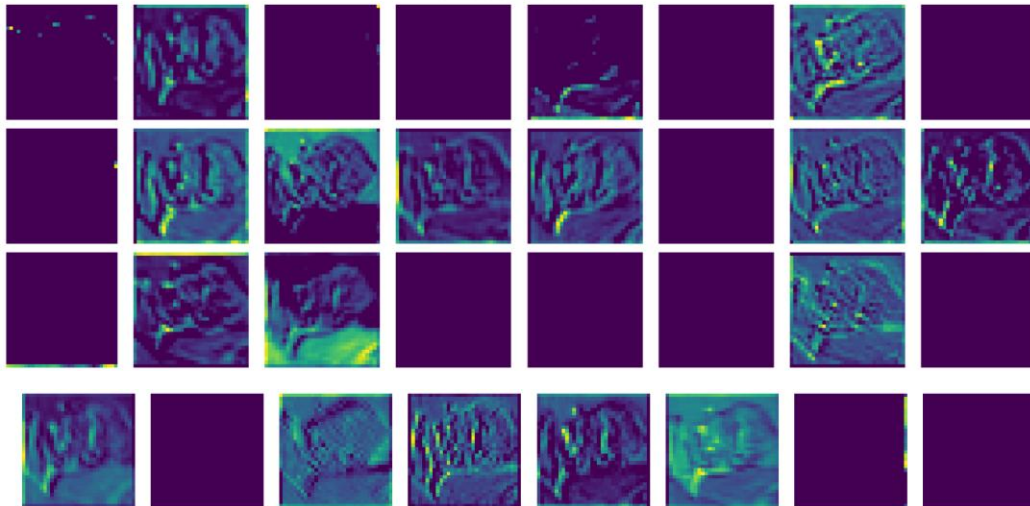
Non-trainable params: 1,408 (5.50 KB)



Sample Predictions (Green=Correct, Red=wrong)

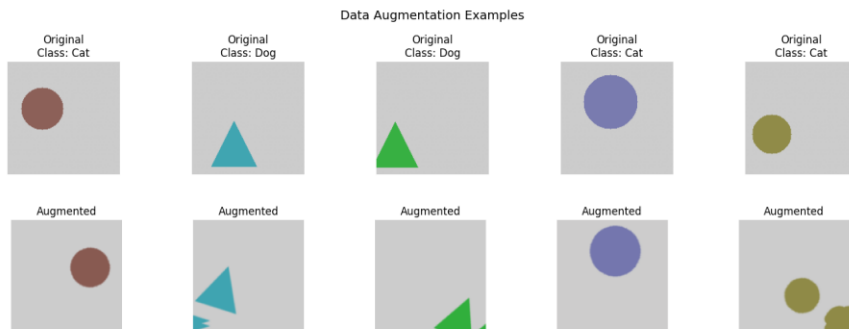
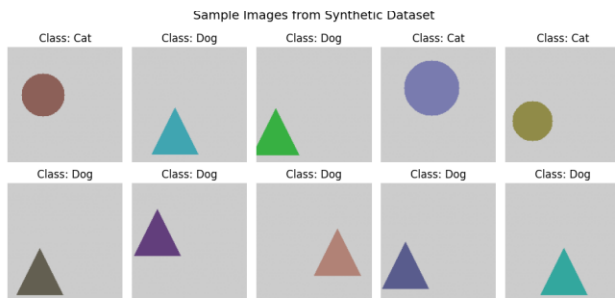


Feature Maps: conv1



LATIHAN 2: TRANSFER LEARNING DENGAN DATA AUGMENTATION

 Training samples: 640
 Validation samples: 160
 Test samples: 200



IMPLEMENTING TRANSFER LEARNING WITH VGG16...
 Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
 58889256/58889256 0s 0us/step
 VGG16 base layers: 19
 VGG16 base frozen for feature extraction

TRANSFER LEARNING MODEL ARCHITECTURE:
 Model: "functional_5"

Layer (type)	Output Shape	Param #	Connected to
input_layer_3 (InputLayer)	(None, 150, 150, 3)	0	-
data_augmentation (Sequential)	(None, 150, 150, 3)	0	input_layer_3[0]-
get_item_0 (GetItem)	(None, 150, 150)	0	data_augmentation.
get_item_1 (GetItem)	(None, 150, 150)	0	data_augmentation.
get_item_2 (GetItem)	(None, 150, 150)	0	data_augmentation.
stack (Stack)	(None, 150, 150, 3)	0	get_item_0[0], get_item_1[0][0], get_item_2[0][0]
add (Add)	(None, 150, 150, 3)	0	stack[0][0]

add (Add)	(None, 150, 150, 3)	0	stack[0][0]
vgg16 (Functional)	(None, 4, 4, 512)	14,714,688	add[0][0]
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0	vgg16[0][0]
dense (Dense)	(None, 256)	131,328	global_average_pooling2d[0][0]
batch_normalization (BatchNormalization)	(None, 256)	1,024	dense[0][0]
dropout (Dropout)	(None, 256)	0	batch_normalization[0][0]
dense_1 (Dense)	(None, 128)	32,896	dropout[0][0]
batch_normalization_1 (BatchNormalization)	(None, 128)	512	dense_1[0][0]
dropout_1 (Dropout)	(None, 128)	0	batch_normalization_1[0][0]
dense_2 (Dense)	(None, 1)	129	dropout_1[0][0]

